

# Lab Session Week 17

## Task 1: Stack

Consider the stack data structure. Assume that you start with an empty stack and that your stack has space for three elements.

1. Execute the following operations in order:

push(A), pop(), pop(), push(C), push(Z), pop(), push(D),  
push(G), push(X), pop(), pop()

Draw the stack after each operation and show which elements are popped or which error messages are issued. Assume that we just move on to the next operation after an error.

2. "Rolling" a certain number of elements on top of the stack is a useful operation. Specifically, consider an algorithm `roll(n, k)` which rotates the first `n` stack elements `k` positions upwards, e.g.

|   |                         |   |
|---|-------------------------|---|
| A |                         | B |
| B | <code>roll(3, 1)</code> | C |
| C | ----->                  | A |
| D |                         | D |

and

|   |                         |   |
|---|-------------------------|---|
| A |                         | C |
| B | <code>roll(3, 2)</code> | A |
| C | ----->                  | B |
| D |                         | D |

Write an algorithm that implements this operation.

3. How does the runtime `roll(n, k)` depend on the size of the stack, `n` and `k`?

## Part 1

To create the example solution, I extend the `Stack` class from Lecture 8 with a string converter for printing the stack.

```
In [39]: class Stack:
         def __init__(self, size):
             self._data = [None] * size
```

```

        self._size = size
        self._top = -1

    def __str__(self):
        """
        Create a string representation of the stack with the bottom to the r

        Fields in the stack are separated by vertical lines. Actual data on
        is shown in black, elements that are currently not part of the stack
        are above the `top` mark of the stack, are shown in yellow using ANSI
        """

        s = 'Bottom| ' + ' | '.join((f"\033[{30 if ix <= self._top else 33}m"
                                     for ix, d in enumerate(self._data))) + ' |'

        # Add top and bottom line with proper length.
        # Subtract 9 for each field to compensate for the color code character
        b = '-' * (len(s) - 9 * len(self._data))
        s = '\n'.join((b, s, b))

        return s

    def empty(self):
        return self._top < 0

    def push(self, x):
        if self._top == self._size - 1:
            raise RuntimeError('Stack overflow')
        self._top += 1
        self._data[self._top] = x

    def pop(self):
        if self.empty():
            raise RuntimeError('Stack underflow')
        self._top -= 1
        return self._data[self._top + 1]

    def top(self):
        if self.empty():
            raise RuntimeError('Stack underflow')
        return self._data[self._top]

```

```

In [40]: s = Stack(4)
         results = []
         print("Start")
         print(s)
         print()

         for func, arg in [("push", "A"),
                           ("pop", None),
                           ("pop", None),
                           ("push", "C"),
                           ("push", "Z"),
                           ("pop", None),
                           ("push", "D"),
                           ("push", "G"),

```

```

        ("push", "X"),
        ("pop", None),
        ("pop", None)]:
    if arg:
        cmd = f's.{func}("{arg}")'
    else:
        cmd = f's.{func}()'
    try:
        r = eval(cmd)
        if r is not None:
            results.append(r)
    except RuntimeError as e:
        r = e
    print(f"{cmd}", end = "")
    if r is not None:
        print(f" -->> {r}")
    else:
        print()
    print(s)
    print()

print("The following data were popped from the stack, in this order:")
print(results)

```

Start

---

|        |      |  |      |  |      |  |      |  |
|--------|------|--|------|--|------|--|------|--|
| Bottom | None |  | None |  | None |  | None |  |
|--------|------|--|------|--|------|--|------|--|

---

s.push("A")

---

|        |   |  |      |  |      |  |      |  |
|--------|---|--|------|--|------|--|------|--|
| Bottom | A |  | None |  | None |  | None |  |
|--------|---|--|------|--|------|--|------|--|

---

s.pop() -->> A

---

|        |   |  |      |  |      |  |      |  |
|--------|---|--|------|--|------|--|------|--|
| Bottom | A |  | None |  | None |  | None |  |
|--------|---|--|------|--|------|--|------|--|

---

s.pop() -->> Stack underflow

---

|        |   |  |      |  |      |  |      |  |
|--------|---|--|------|--|------|--|------|--|
| Bottom | A |  | None |  | None |  | None |  |
|--------|---|--|------|--|------|--|------|--|

---

s.push("C")

---

|        |   |  |      |  |      |  |      |  |
|--------|---|--|------|--|------|--|------|--|
| Bottom | C |  | None |  | None |  | None |  |
|--------|---|--|------|--|------|--|------|--|

---

s.push("Z")

---

|        |   |  |   |  |      |  |      |  |
|--------|---|--|---|--|------|--|------|--|
| Bottom | C |  | Z |  | None |  | None |  |
|--------|---|--|---|--|------|--|------|--|

---

s.pop() -->> Z

---

|        |   |  |   |  |      |  |      |  |
|--------|---|--|---|--|------|--|------|--|
| Bottom | C |  | Z |  | None |  | None |  |
|--------|---|--|---|--|------|--|------|--|

---

s.push("D")

---

|        |   |  |   |  |      |  |      |  |
|--------|---|--|---|--|------|--|------|--|
| Bottom | C |  | D |  | None |  | None |  |
|--------|---|--|---|--|------|--|------|--|

---

s.push("G")

---

|        |   |  |   |  |   |  |      |  |
|--------|---|--|---|--|---|--|------|--|
| Bottom | C |  | D |  | G |  | None |  |
|--------|---|--|---|--|---|--|------|--|

---

s.push("X")

---

|        |   |  |   |  |   |  |   |  |
|--------|---|--|---|--|---|--|---|--|
| Bottom | C |  | D |  | G |  | X |  |
|--------|---|--|---|--|---|--|---|--|

---

s.pop() -->> X

---

|        |   |  |   |  |   |  |   |  |
|--------|---|--|---|--|---|--|---|--|
| Bottom | C |  | D |  | G |  | X |  |
|--------|---|--|---|--|---|--|---|--|

---

s.pop() -->> G

-----  
Bottom| C | D | G | X |  
-----

The following data were popped from the stack, in this order:  
['A', 'Z', 'X', 'G']

## Part 2

### Roll by a single position ( $k = 1$ )

The `Roll_1(S, n)` algorithm shall rotate the `n` elements on the top of the stack `S` upwards by one position, placing the element at the top at the  $n^{\text{th}}$  position from the top. The basic idea of the algorithm thus is

1. Copy the top element to a temporary variable.
2. Move the next `n-1` elements up one position each.
3. Put the element that was originally on top into the `n`-th position from top.

Note that we always need to think from the top element of the stack downwards. The loop variable `j` in the following code shows "distance from top element" and therefore runs from zero.

We can implement this as follows:

```
Roll_1(S, n)
    if n < 2
        error "Must roll at least two elements"
    if S.top < n
        error "Too few elements on stack"

    t = S[S.top]
    for j = 0 .. n - 2
        to = S.top - j
        from = to - 1
        S[to] = S[from]
    S[S.top - n + 1] = t
```

I introduced the variables `to` and `from` here to make the code more readable. By plugging in, we can check that `to` will range from `S.top` for `j=0` to `S.top - n + 2` for `j=n-2` and `from` will correspondingly run from `S.top - 1` to `S.top - n + 1`, and the loop moves `n - 1` elements. In the last round through the loop, we have `to == S.top - n + 2`. The element below that has index `S.top - n + 1` and the last line stores in the value saved in `t` in that location.

### Roll by $k$ positions, simple solution

We simply roll  $k$  times by one position.

```

Roll(S, n, k)
  for j = 1 .. k
    Roll_1(S, n)

```

## Roll by $k$ positions, efficient solution

We apply the same logic as above, but now we need to store the top  $k$  elements in a temporary array. We then move the next  $n-k$  elements upwards by  $k$  positions, before inserting the data from temporary storage below them. The loop variable  $j$  again indicates "distance from top".

```

Roll(S, n, k)
  if n < 2
    error "Must roll at least two elements"
  if S.top < n
    error "Too few elements on stack"
  if k >= n
    error "k < n required"

  T = Array of size k
  for j = 0 .. k-1
    T[j] = S[S.top - j]

  for j = 0 .. n - k - 1
    to = S.top - j
    from = to - k
    S[to] = S[from]

  first_to_write_back_to = S.top - n + k
  for j = 0 .. k - 1
    to = first_to_write_back_to - j
    S[to] = T[k]

```

The second for loop moves data to positions  $S.top$  (for  $j=0$ ) to  $S.top-n+k+1$ , i.e., moves  $n-k$  elements. Each element is filled with the value found  $k$  positions further down in the stack, and the last element that will be moved up is at position  $from = S.top - (n-k-1) - k = S.top - n + 1$ , which is the  $n$ -th element on the stack from the top as expected. Since that for loop moved data into positions down to  $S.top-n+k+1$ , the data from the temporary storage  $T$  needs to be written to positions starting one further down, i.e., starting at  $first\_to\_write\_back\_to = S.top - n + k$ . The third loop then writes this data back from  $T$ .

## Part 3

### Roll\_1

The loop is executed  $n-1$  times, while error checking and copying to/from  $t$  takes constant time. We thus have runtime  $\Theta(n)$  and require only a single variable of extra

storage.

## Simple Roll( $n, k$ )

We execute `Roll_1`  $k$  times. Since each execution of that algorithm is  $\Theta(n)$ , the total runtime will be  $\Theta(kn)$ . Note in particular that each element that is rotated will be copied  $k$  times, once for each call to `Roll_1()`. If we want bounds only in  $n$ , we clearly have as best case  $k = 1$  and as worst case  $k = n - 1$ , which yields  $\Omega(n)$  as lower and  $O(n^2)$  as upper bound. Extra storage is a single variable as for `Roll_1(n)`.

## Efficient Roll( $n, k$ )

We focus on the copy operations. The first loop performs  $k$  copies, the second loop  $n - k$  copies and the third loop  $k$  copies. We thus have in total  $n + k$  copy operations and the runtime will be  $\Theta(n + k)$ . For bounds expressed just in  $n$ , because  $1 \leq k < n$ , we have  $\Omega(n)$  and  $O(n)$ . The price we have to pay for this improvement in runtime is the need for  $\Theta(k)$  extra storage for the temporary data `T`. It is common to find such space-time trade-offs when optimizing algorithms.

## Task 2: Queue

Consider the queue data structure, starting with a queue that can hold three data items and is empty to begin with.

1. If you implement the queue using an array, how many elements does the array need to have?
2. Execute the following operations in order (abbreviating here "enqueue" as `enq` and "dequeue" as `deq`):

```
enq(A), deq(), deq(), enq(Z), enq(F), enq(Q), enq(Y), deq(),  
deq() enq(H), deq()
```

Draw the queue after each operation, including pointers to head and tail, and show elements that are dequeued or error messages issued. Assume that we just move on to the next operation after an error.

### Part 1

We always keep one element empty, so that "head == tail" indicates an empty queue, while "tail + 1 == head" indicates a full queue. Therefore, the array used to store the queue must have one more element than the largest number of elements we want to store.

### Part 2

This is the `Queue` class as in Lecture 8, with two modifications:

- The constructor now takes the `capacity` of the queue as argument, i.e., the maximum number of elements we want to be able to store.
- The string converter is similar to the one for the `Stack` class above. The implementation is a bit more complicated as it also shows "head" and "tail" markers and because checking whether an element is between head and tail, i.e., is active, needs to take the "wrap-around" into consideration.

```
In [60]: class Queue:
    def __init__(self, capacity):
        """capacity - number of elements queue can hold"""

        self.size = capacity + 1
        self._data = [None] * self.size
        self._head = 0
        self._tail = 0

    def __str__(self):
        """
        Create a string representation of the stack with the bottom to the r

        Fields in the stack are separated by vertical lines. Actual data on
        is shown in black, elements that are currently not part of the stack
        are above the `top` mark of the stack, are shown in yellow using ANSI
        """

        s = '|' + ' | '.join((f"\033[{30 if self._active(ix) else 33}m{str(d)}"
                               for ix, d in enumerate(self._data))) + ' |'

        # Add top and bottom line with proper length.
        # Subtract 9 for each field to compensate for the color code character
        b = '-' * (len(s) - 9 * len(self._data))

        # Build a line with markers for Head and Tail. We need to adjust to
        # fields in the queue. For this, we need a version of the "queue str
        # contain the extra codes controlling color.
        s_plain = '|' + ' | '.join((f"{str(d)}" for ix, d in enumerate(self._data)))
        ht = [' '] * len(b)
        ctr = -1
        for ix, c in enumerate(s_plain):
            if c == '|':
                ctr += 1
                if self._head == ctr:
                    ht[ix + 1] = "H"
                if self._tail == ctr:
                    ht[ix + 3] = "T"

        s = '\n'.join((b, s, b, ''.join(ht)))

        return s

    def _active(self, ix):
```



```

        """Check if index is between head and tail, taking wrap-around into

        if self._tail < self._head:
            return ix < self._tail or self._head <= ix
        else:
            return self._head <= ix < self._tail

    def enqueue(self, x):
        new_tail = (self._tail + 1) % self.size
        if new_tail == self._head:
            raise RuntimeError('Queue is full')
        self._data[self._tail] = x
        self._tail = new_tail

    def dequeue(self):
        if self._head == self._tail:
            raise RuntimeError('Queue empty')
        x = self._data[self._head]
        self._head = (self._head + 1) % self.size
        return x

```

```

In [62]: s = Queue(3)
results = []
print("Start")
print(s)
print()

for func, arg in [("enqueue", "A"),
                  ("dequeue", None),
                  ("dequeue", None),
                  ("enqueue", "Z"),
                  ("enqueue", "F"),
                  ("enqueue", "Q"),
                  ("enqueue", "Y"),
                  ("dequeue", None),
                  ("dequeue", None),
                  ("enqueue", "H"),
                  ("dequeue", None)]:

    if arg:
        cmd = f's.{func}("{arg}")'
    else:
        cmd = f's.{func}()'
    try:
        r = eval(cmd)
        if r is not None:
            results.append(r)
    except RuntimeError as e:
        r = e
    print(f"{cmd}", end = "")
    if r is not None:
        print(f" -->> {r}")
    else:
        print()
print(s)
print()

```

```
print("The following data were dequeued, in this order:")  
print(results)
```

Start

---

|  |      |      |      |      |
|--|------|------|------|------|
|  | None | None | None | None |
|--|------|------|------|------|

---

H T

s.enqueue("A")

---

|   |      |      |      |
|---|------|------|------|
| A | None | None | None |
|---|------|------|------|

---

H T

s.dequeue() -->> A

---

|   |      |      |      |
|---|------|------|------|
| A | None | None | None |
|---|------|------|------|

---

H T

s.dequeue() -->> Queue empty

---

|   |      |      |      |
|---|------|------|------|
| A | None | None | None |
|---|------|------|------|

---

H T

s.enqueue("Z")

---

|   |   |      |      |
|---|---|------|------|
| A | Z | None | None |
|---|---|------|------|

---

H T

s.enqueue("F")

---

|   |   |   |      |
|---|---|---|------|
| A | Z | F | None |
|---|---|---|------|

---

H T

s.enqueue("Q")

---

|   |   |   |   |
|---|---|---|---|
| A | Z | F | Q |
|---|---|---|---|

---

T H

s.enqueue("Y") -->> Queue is full

---

|   |   |   |   |
|---|---|---|---|
| A | Z | F | Q |
|---|---|---|---|

---

T H

s.dequeue() -->> Z

---

|   |   |   |   |
|---|---|---|---|
| A | Z | F | Q |
|---|---|---|---|

---

T H

s.dequeue() -->> F

---

```
| A | Z | F | Q |
```

```
T           H
```

```
s.enqueue("H")
```

```
| H | Z | F | Q |
```

```
T           H
```

```
s.dequeue() -->> Q
```

```
| H | Z | F | Q |
```

```
H           T
```

The following data were dequeued, in this order:  
['A', 'Z', 'F', 'Q']

## Task 3: Linked lists

Assume that we have data in a linked list. We now want to sort this data. You can use any of the sort algorithms we have discussed, or any other you know of.

- How would you need to adapt the sort algorithm to work for the linked list?
- What is the key problem in applying the sort algorithms we developed for array to linked lists?
- Are there some sort algorithms that are more easily adapted to lists than others?
- Is there a difference between single- and double-linked lists?

## General insights

All sorting algorithms we have seen so far rely on random access to the underlying data, as we have it for arrays. If we have an array `A`, we can easily access any element `A[i]`, both to check or change its value. In a linked list, on the other hand, we can only access elements in order, starting from the head of the list. Therefore, we cannot directly apply existing sorting algorithms to linked lists.

## A straightforward algorithm

Since we can easily traverse a list from beginning to end, algorithms based on moving through the list from start to end comparing only neighboring elements and swapping them if they are in incorrect order is straightforward to implement. Moving through the list once will guarantee that the largest element in the list will be at the end of the list afterwards. The next traversal will ensure that the last two elements are sorted, and so on. In this process, we need to count the length of the list, so we know when we are done. This can be implemented as follows:

```

01: ListSort(L)
02:     if L.head = NIL
03:         return
04:
05:     to_sort = Infinity
06:
07:     while to_sort > 1
08:         x = L.head
09:         j = 1
10:
11:         while x.next != NIL and j < to_sort
12:             if x.key > x.next.key
13:                 SwapWithNext(L, x)
14:             else
15:                 x = x.next
16:                 j = j + 1
17:
18:         to_sort = j - 1

```

The algorithm works as follows:

1. We check for an empty list to avoid corner cases below. We can now be sure that the list has at least one element.
2. We don't know how many elements the list has. To avoid spending extra time on counting the number of elements, we assume infinity to begin with. We will find the right number during the first traversal.
3. The outer loop manages repeated traversals, as long as there is something to do.
4. For each traversal, we start from the head. `j` is the number of the next element we look at, counted starting with head.
5. In the condition on line 11, the first condition makes sure we do not run over the end of the list. This is only relevant for the first traversal. The second condition makes sure we stop once we have traversed all data not yet sorted.
6. Lines 13 swaps `x` with its successor if necessary. After this swap, `x` and `x.next` have traded places in the list, i.e., the list entry to which `x` refers is now one element further from head than before the swap. Therefore, we need to execute line 15 only if we did not swap the elements.
7. The inner loop makes sure that after each iteration, `x` contains the largest element between head and `x` in the list.
8. When the inner loop terminates, we therefore have only the elements from head to the predecessor of `x`, i.e., `j-1` elements left to sort and we update `to_sort` correspondingly.

The runtime of this algorithm is  $\Theta(n^2)$  due to the nested lists, where  $n$  is then number of elements in the list.

## Adapting merge sort

We can adapt merge sort for list a list `L` as follows.

We first define a wrapper that measures the length of the list and then delegates the work to a recursive function:

```
MergeSortList(L)
    n = 0
    x = L.head
    while x != NIL
        n = n + 1
        x = x.next
    if n < 2
        return

    L = DoMergeSortList(L, n)
    return L
```

Here the actual worker:

```
01: DoMergeSortList(L, n)
02:     if n < 2
03:         return L
04:
05:     m = floor(n / 2)
06:     A.head = L.head
07:     x = L.head
08:     for j = 1..m
09:         x = x.next
10:     B.head = x.next
11:     x.next = NIL
12:
13:     A = DoMergeSortList(A, m)
14:     B = DoMergeSortList(B, n - m)
15:
16:     C = empty list
17:     if A.head.key < B.head.key
18:         C.head = A.head
19:         List-Delete(A, A.head)
20:     else
21:         C.head = B.head
22:         List-Delete(B, B.head)
23:     C.head.prev = NIL
24:     C.head.next = NIL
25:
26:     x = C.head
27:     while A.head != NIL and B.head != NIL
28:         if A.head.key < B.head.key
29:             q = A.head
30:             List-Delete(A, A.head)
31:         else
32:             q = B.head
```

```

33:         List-Delete(B, B.head)
34:     x.next = q
35:     q.prev = x
36:     q.next = NIL
37:     x = q
38:
39:     if A.head != NIL
40:         q.next = A.head
41:     else
42:         q.next = B.head
43:     q.next.prev = q
44:
45:     return C

```

The key points here are as follows:

1. On lines 5–11 we split list **L** into two lists **A** and **B** of lengths **m** and **N-m**, respectively. Because of the check on lines 2–3, we are certain that both lists have at least one element.
2. Lines 13–14 sort these two lists recursively. After these calls complete, the lists **A** and **B** are sorted.
3. Lines 16–24 create a new list **C** to hold the result, check which list contains the smallest element and then move that list from its current list to the head of list **C**.
4. On lines 26–37, we first set **x** to the only element in list **C**. We then traverse **A** and **B** in the while loop, always taking the smaller of the two head elements and appending it to **C**. Since we work with lists, we can actually *move* elements between lists instead of copying to extra storage. Line 37 ensures that **x** always points to the last element in **C**.
5. We leave the while loop as soon as one of the two lists is exhausted. Because of the way in which the loop is designed and because we know that both lists contain at least one element when we first enter the while-loop, we can be certain that one of the two lists will be non-empty when we leave the loop.
6. We then simply append the list still containing elements at the end of **C**.
7. Finally, we return **C**, which essentially means returning the head.

While technicalities differ and we need to measure the length of the list once at the beginning (in time  $\Theta(n)$ ), the overall structure is the same as for the array-based merge sort algorithm. Therefore, the runtime analysis for that algorithm carries over and we have  $T(n) = \Theta(n \lg n)$ . Interestingly, since we can move elements between lists, we avoid the memory overhead we encountered for array-based merge sort.

## Part 4

When working with singly-linked lists, the following changes apply:

- For the simple algorithm, we need to keep track of the predecessor of `x` explicitly and pass that to a modified swap function.
- The merge sort algorithm becomes simpler: We don't need anything extra to delete the head element of a list, and we can remove lines 35 and 43, which update the `prev` element.

In [ ]: